

YaniG Documentation

Table of Contents

1. [Introduction / Overview](#)
2. [Setup & Installation](#)
3. [Architecture & Design](#)
4. [API Reference](#)
5. [Examples & Use Cases](#)
6. [FAQ / Troubleshooting](#)
7. [Contributing / Development](#)

Diagrams

- [Class Diagram](#)
 - [Data Flow Diagram](#)
-

1. Introduction / Overview

Project Purpose and Motivation

YaniG ("Yet Another Natural Intelligence Gedöns") is an educational framework designed to teach students about multimodal generative AI models and their orchestration. The term "Gedöns" is German for "a collection of things" or "paraphernalia," reflecting the framework's nature as a toolkit of components for working with various AI capabilities.

The primary motivation behind YaniG is to provide a structured, accessible way for students to understand and experiment with modern AI technologies in an educational context. By offering a modular architecture with clear interfaces, YaniG makes it easier to:

- Understand how different AI models can be integrated into applications
- Learn about workflow orchestration for AI components
- Experiment with multimodal inputs and outputs (text, embeddings, audio)
- Implement standardized function calls for language models

Application Areas

YaniG is particularly suited for the following application areas:

1. **Educational Settings:**

- 2. University courses on AI and machine learning
- 3. Workshops and tutorials on generative AI
- 4. Self-guided learning about AI orchestration

5. **Prototyping and Experimentation:**

- 6. Quick implementation of AI-powered features
- 7. Testing different AI models with the same interface
- 8. Building proof-of-concept applications

9. **Research:**

- 10. Comparing different AI models and approaches
- 11. Developing new components for specialized tasks
- 12. Benchmarking performance across implementations

13. **Demonstration:**

- 14. Showcasing AI capabilities through simple examples
- 15. Illustrating concepts like function calling in language models
- 16. Presenting multimodal AI workflows

Technological Context

YaniG operates within the rapidly evolving landscape of generative AI and multimodal systems. Key technological aspects include:

Multimodal Generative AI

The framework supports multiple modalities through specialized components: - **Text Generation:** Integration with various LLMs (Large Language Models) through providers like OpenAI, HuggingFace, and Ollama - **Embeddings:** Vector representations of text for semantic search and comparison - **Audio:** Text-to-speech capabilities through ElevenLabs - **Document Processing:** PDF reading and analysis

Workflow Orchestration

YaniG implements a component-based architecture that allows for: - **Modular Design:** Each AI capability is encapsulated in a component with a standardized interface - **Component Chaining:** Components can be connected to create complex workflows - **Standardized Data Exchange:** Components communicate through a common ComponentResultObject format - **Extensibility:** New components can be added to support additional AI capabilities

Function Calling Standardization

The MCP-Server application demonstrates how language models can be used to standardize function calls, allowing for: - **Consistent Interfaces:** Uniform way to interact with different AI models - **Dynamic Resource Allocation:** Adding new capabilities at runtime - **Structured Responses:** Formatting model outputs in a consistent way

Framework Goals and Target Audience

Primary Goals

1. Provide a learning platform for understanding AI orchestration
2. Demonstrate integration patterns for various AI models
3. Offer a modular, extensible architecture for experimentation
4. Simplify the process of working with multiple AI modalities

Target Audience

1. **Students:** Learning about AI integration and application development
2. **Educators:** Teaching AI concepts through practical examples
3. **Developers:** Experimenting with AI capabilities in a structured environment
4. **Researchers:** Testing and comparing different AI approaches

YaniG strikes a balance between simplicity for beginners and flexibility for more advanced users, making it suitable for a wide range of educational scenarios while providing enough depth for meaningful experimentation and learning.

2. Setup & Installation

Prerequisites

Before installing YaniG, ensure your system meets the following requirements:

- **Python:** Version 3.12.x or higher
- **Operating System:** Compatible with Windows, macOS, and Linux
- **Package Manager:** pip (included with Python)
- **Optional:** Node.js (for MCP-Server functionality)

Installation Instructions

Basic Installation

YaniG can be installed directly from the GitHub repository:

```
# Clone the repository
git clone https://github.com/spie-mkroehn/yanig.git

# Navigate to the project directory
cd yanig

# Create a virtual environment (recommended)
python -m venv venv

# Activate the virtual environment
# On Windows:
venv\Scripts\activate
# On macOS/Linux:
source venv/bin/activate

# Install dependencies
pip install -r requirements.txt
```

Environment Setup

YaniG uses environment variables for configuration. Create a `.env` file in the project root with the following settings:

```
# API Keys for various services
OPENAI_API_KEY=your_openai_api_key
```

```
HUGGINGFACEHUB_API_TOKEN=your_huggingface_token  
ELEVENLABS_API_KEY=your_elevenlabs_key
```

Adjust these settings according to your specific requirements and available API keys.

MCP-Server Setup

If you want to use the Model Context Protocol server functionality:

```
# Navigate to the MCP-Server directory  
cd applications/mcpserver  
  
# Install Node.js dependencies (requires npm)  
npm install  
  
# Run the server  
python server.py
```

Quick Start Guide

Here's a minimal example to get you started with YaniG, based on the examples provided in the repository:

Basic PDF Reader Example

This example demonstrates how to extract text content from a PDF document. The PDFReaderComponent allows you to specify which pages to read and returns structured content including chapter information and page numbers along with the extracted text.

```
from core import ComponentResultObject  
from components import PDFReaderComponent  
from os.path import join  
  
# Initialize the PDF reader component  
pdfreader = PDFReaderComponent()  
  
# Create input with source path and page settings  
input = ComponentResultObject()  
input["source"] = join("data", "pdf", "beispiel.pdf")  
input["content"] = {}  
input["content"]["page_number"] = 1  
input["content"]["page_count"] = 1  
  
# Process the PDF and get results  
results = pdfreader.invoke([input])
```

```
# Print the extracted content
for result in results:
    print(result["content"]["chapter"], result["content"]["page_number"])
    print(result["content"]["original_text"])
```

Embedding Generation Example

This example shows how to generate vector embeddings from text. Embeddings are numerical representations of text that capture semantic meaning, allowing for operations like semantic search and similarity comparison. The EmbeddingComponent converts text into high-dimensional vectors that can be used for various AI applications.

```
from core import ComponentResultObject
from components import EmbeddingComponent

# Initialize the embedding component
ec = EmbeddingComponent()

# Create input with text to embed
input = ComponentResultObject()
input["content"] = {}
input["content"]["original_text"] = "This is a sample text for embedding"

# Generate embeddings
results = ec.invoke([input])

# Use the embeddings for further processing
print(f"Generated embedding with {len(results[0]['content']['embedding'])}
dimensions")
```

Simple Chat Example

This example demonstrates how to use the ChatComponent to interact with language models. The component maintains conversation context between interactions, allowing for multi-turn conversations. You can provide simple text prompts and receive natural language responses from the underlying AI model.

```
from components import ChatComponent

# Initialize the chat component
chat = ChatComponent()

# Start a conversation
response = chat.invoke("Tell me about natural language processing")
print(response)
```

```
# Continue the conversation
```

```
follow_up = chat.invoke("What are some practical applications?")
```

```
print(follow_up)
```

Getting Started with Your Own Project

To integrate YaniG into your own projects:

1. Import the necessary components from the appropriate modules
2. Initialize the components with desired configurations
3. Create input objects using the ComponentResultObject format
4. Process inputs through component invoke() methods
5. Chain components together for more complex workflows

For more detailed examples, refer to the Examples & Use Cases section of this documentation or examine the `examples.py` file in the repository.

Troubleshooting Installation

If you encounter issues during installation:

- Ensure Python version compatibility
- Check that all required dependencies are installed
- Verify API keys are correctly set in the `.env` file
- For model-specific issues, consult the documentation of the respective model provider

For more detailed troubleshooting, see the FAQ section of this documentation.

3. Architecture & Design

System Overview

YaniG is designed as a modular framework for orchestrating multimodal AI components. The architecture follows a component-based design pattern that enables flexibility, extensibility, and clear separation of concerns. This section provides a comprehensive overview of the system architecture, key components, and their interactions.

High-Level Architecture

At its core, YaniG consists of several key modules organized in a layered architecture:

1. **Core Layer:** Provides fundamental structures and utilities
2. Component result objects
3. Component chaining mechanism
4. Configuration settings
5. Structured response handling
6. **Components Layer:** Implements the actual processing units
7. Base component definitions
8. Specialized components for different tasks
9. Component lifecycle management
10. **Functions Layer:** Integrates with external AI services
11. Model-specific implementations
12. Provider integrations (OpenAI, HuggingFace, Ollama)
13. Audio processing capabilities
14. **API Layer:** Provides interfaces for external communication
15. Vector database API
16. Component Result Object API
17. External service integrations
18. **Agents Layer:** Implements autonomous agents
19. Agent definitions and behaviors
20. Task planning and execution
21. Multi-agent coordination
22. **Applications Layer:** Demonstrates practical implementations
23. Example applications
24. MCP-Server for function call standardization

This layered approach allows for clear separation of concerns while maintaining flexibility for extension and customization.

Core Components

Component Base Classes

The foundation of YaniG's architecture is the component system, which starts with the `BaseComponent` class:

```
class BaseComponent(ABC, BaseModel):  
    """  
    This is somehow the "contract" for each component  
    has no other functionality yet  
    """  
    @abstractmethod  
    def invoke(self, input: List[ComponentResultObject]) ->  
    List[ComponentResultObject]:  
        pass
```

This abstract base class defines the fundamental contract that all components must adhere to. It establishes:

- A standardized `invoke()` method signature
- Input/output typing using `ComponentResultObject`
- Integration with Pydantic's `BaseModel` for validation

For components that require streaming capabilities, the framework provides `BaseStreamingComponent`:

```
class BaseStreamingComponent(BaseComponent):  
    """  
    Base class for components that support streaming responses  
    """  
    @abstractmethod  
    def invoke_streaming(self, input: ComponentResultObject) ->  
    Iterator[ComponentResultObject]:  
        pass
```

This extension adds streaming functionality while maintaining compatibility with the base component interface.

Component Result Objects

The `ComponentResultObject` serves as the standardized data container for passing information between components:

```

class ComponentResultObject(dict):
    """
    A dictionary-based object for storing component results
    Provides a standardized structure for data exchange
    """
    def __init__(self, *args, **kwargs):
        super().__init__(*args, **kwargs)
        # Initialize standard fields if not present
        if "preprocessing" not in self:
            self["preprocessing"] = {}
        if "content" not in self:
            self["content"] = {}

```

This object: - Extends Python's dictionary type for flexibility - Provides standardized fields for content and preprocessing metadata - Enables consistent data exchange between heterogeneous components

Component Chain

The `ComponentChain` class enables the orchestration of multiple components in sequence:

```

class ComponentChain:
    """
    Chains multiple components together for sequential processing
    """
    def __init__(self, components: List[BaseComponent]):
        self.components = components

    def process(self, initial_input: ComponentResultObject) ->
    List[ComponentResultObject]:
        current_input = [initial_input]
        for component in self.components:
            current_input = component.invoke(current_input)
        return current_input

```

This orchestration mechanism: - Takes a list of components during initialization - Processes input through each component sequentially - Passes the output of each component as input to the next - Returns the final processed results

Module Relationships

The YaniG framework is organized into several interconnected modules:

Core Module

The `core` module provides fundamental structures and utilities: -

`ComponentResultObject` : Data container for component communication -

`ComponentChain` : Orchestration mechanism for component sequences - `settings` :

Configuration management - `structuredresponses` : Standardized response formatting

Core Module

Components Module

The `components` module implements the processing units: - `BaseComponent` :

Abstract base class for all components - `BaseStreamingComponent` : Extension for

streaming capabilities - Specialized components: - `PDFReaderComponent` : Extracts and processes PDF content - `ChatComponent` : Interfaces with chat models -

`EmbeddingComponent` : Generates vector embeddings - `ComparatorComponent` : Compares and ranks content

Components Module

Functions Module

The `functions` module integrates with external AI services: - Base implementations: -

`basechat.py` : Abstract interface for chat models - `baseembedding.py` : Abstract

interface for embedding models - `basereranker.py` : Abstract interface for reranking

models - Provider-specific implementations: - OpenAI: `openaichat.py` ,

`openaiembedding.py` , `openaireranker.py` - HuggingFace: `huggingfacechat.py` ,

`huggingfaceembedding.py` , `huggingfacreranker.py` - Ollama: `ollamachat.py` ,

`ollamaembedding.py` , `ollamareranker.py` - Additional services: - `elevenlabsaudio.py` :

Text-to-speech capabilities

Functions Module

API Module

The `api` module provides interfaces for external communication: - `cro.py` : Component

Result Object API for data exchange - `vectordb.py` : Vector database interface for storing

and retrieving embeddings - `serviceintegration.py` : Integration with external services

and APIs - `dataprocessing.py` : Data processing and transformation utilities

API Module

Agents Module

The `agents` module implements autonomous agents: - `baseagent.py` : Abstract base class for all agents - `taskplanner.py` : Planning and scheduling of agent tasks - `executor.py` : Execution of agent tasks and actions - `multiagent.py` : Coordination between multiple agents - `memory.py` : Agent memory and knowledge management

Agents Module

Applications Module

The `applications` module demonstrates practical implementations: - `examples.py` : Sample usage patterns - `mcpserver` : Model Context Protocol server implementation

Applications Module

Data Flow

The typical data flow in a YaniG application follows these steps:

1. **Initialization:** Components are instantiated with appropriate configurations
2. **Input Preparation:** Input data is wrapped in a `ComponentResultObject`
3. **Processing:** Data is processed through one or more components
4. **Transformation:** Each component transforms the input according to its functionality
5. **Result Collection:** Final results are collected from the last component
6. **Utilization:** Results are used by the application as needed

For more complex workflows, components can be chained together using the `ComponentChain` class, allowing for sophisticated processing pipelines.

YaniG Data Flow Diagram

Design Patterns

YaniG implements several key design patterns:

1. **Abstract Factory:** The base classes in the functions module serve as abstract factories for creating specific implementations.
2. **Strategy Pattern:** Different component implementations provide alternative strategies for the same interface.

3. **Adapter Pattern:** Components adapt between different AI service APIs and the standardized YaniG interface.
4. **Facade Pattern:** The high-level components provide simplified interfaces to complex subsystems.
5. **Chain of Responsibility:** The component chain passes requests along a chain of handlers.

These patterns contribute to the framework's flexibility, extensibility, and maintainability.

Extension Points

YaniG is designed to be extended in several ways:

1. **New Components:** Create new component classes by extending `BaseComponent`
2. **New Model Integrations:** Implement new model providers by extending the base classes in the functions module
3. **Custom Applications:** Build applications using the existing components or extend with custom ones
4. **Enhanced Workflows:** Create complex processing pipelines using `ComponentChain`

This extensibility allows YaniG to adapt to new AI capabilities and use cases as they emerge.

4. API Reference

This section provides detailed documentation for all modules, classes, methods, and functions in the YaniG framework. The API is organized by module, with each component and function described in detail.

Core Module

The core module provides the fundamental structures and utilities that power the YaniG framework.

ComponentResultObject

```
class ComponentResultObject(dict)
```

A dictionary-based object for storing and passing data between components.

Methods

Method	Description
<code>__init__(*args, **kwargs)</code>	Initializes a new ComponentResultObject with standard fields.

Standard Fields

Field	Type	Description
<code>source</code>	str	Source of the data (e.g., file path, URL)
<code>content</code>	dict	Main content container for the component data
<code>preprocessing</code>	dict	Metadata about preprocessing steps
<code>target</code>	str	Target destination for the data (optional)

Usage Example

```
# Create a new ComponentResultObject
result = ComponentResultObject()
result["source"] = "data/example.txt"
result["content"] = {"original_text": "Sample text"}
```

ComponentChain

```
class ComponentChain
```

Chains multiple components together for sequential processing.

Methods

Method	Description
<code>__init__(components: List[BaseComponent])</code>	Initializes a new ComponentChain with a list of components.
<code>process(initial_input: ComponentResultObject) -> List[ComponentResultObject]</code>	Processes the input through all components in sequence.

Usage Example

```
# Create a chain of components
pdf_reader = PDFReaderComponent()
embedding = EmbeddingComponent()
chain = ComponentChain([pdf_reader, embedding])

# Process input through the chain
input_obj = ComponentResultObject()
input_obj["source"] = "document.pdf"
results = chain.process(input_obj)
```

settings

Module for managing configuration settings across the framework.

Functions

Function	Description
<code>load_dotenv(dotenv_path: str = None)</code>	Loads environment variables from .env file.
<code>get_setting(key: str, default: Any = None) -> Any</code>	Retrieves a setting value by key.

structuredresponses

Module for handling structured responses from components.

Functions

Function	Description
<code>format_response(data: dict, template: str) -> str</code>	Formats data according to a template.
<code>parse_structured_output(text: str, schema: dict) -> dict</code>	Parses structured text into a dictionary based on schema.

Components Module

The components module contains the processing units that perform specific tasks within the framework.

BaseComponent

```
class BaseComponent(ABC, BaseModel)
```

Abstract base class that defines the interface for all components.

Methods

Method	Description
<code>invoke(input: List[ComponentResultObject]) -> List[ComponentResultObject]</code>	Abstract method that processes input and returns results.

BaseStreamingComponent

```
class BaseStreamingComponent(BaseComponent)
```

Base class for components that support streaming responses.

Methods

Method	Description
<code>invoke(input: List[ComponentResultObject]) -> List[ComponentResultObject]</code>	Processes input and returns results (inherited from BaseComponent).

Method	Description
<code>invoke_streaming(input: ComponentResultObject) -> Iterator[ComponentResultObject]</code>	Abstract method that processes input and yields results incrementally.

PDFReaderComponent

```
class PDFReaderComponent(BaseComponent)
```

Component for reading and extracting content from PDF files.

Methods

Method	Description
<code>__init__()</code>	Initializes a new PDFReaderComponent.
<code>invoke(input: List[ComponentResultObject]) -> List[ComponentResultObject]</code>	Processes PDF files and extracts content.

Input Fields

Field	Type	Description
<code>source</code>	str	Path to the PDF file
<code>content.page_number</code>	int	Page number to extract (1-based)
<code>content.page_count</code>	int	Number of pages to extract

Output Fields

Field	Type	Description
<code>content.original_text</code>	str	Extracted text content
<code>content.chapter</code>	str	Chapter information (if available)
<code>content.page_number</code>	int	Page number of the extracted content

ChatComponent

```
class ChatComponent(BaseComponent)
```

Component for interacting with chat models.

Methods

Method	Description
<code>__init__(model: str = None, provider: str = None)</code>	Initializes a new ChatComponent with optional model and provider.
<code>invoke(input: Union[str, List[ComponentResultObject]]) -> Union[str, List[ComponentResultObject]]</code>	Processes text input and returns chat responses.

Input Fields

Field	Type	Description
<code>content.original_text</code>	str	Text prompt for the chat model
<code>content.chat_history</code>	list	Previous conversation history (optional)
<code>preprocessing.system_prompt</code>	str	System prompt for the chat model (optional)

Output Fields

Field	Type	Description
<code>content.result_text</code>	str	Response from the chat model
<code>content.chat_history</code>	list	Updated conversation history

EmbeddingComponent

```
class EmbeddingComponent(BaseComponent)
```

Component for generating vector embeddings from text.

Methods

Method	Description
<code>__init__(model: str = None, provider: str = None)</code>	Initializes a new EmbeddingComponent with optional model and provider.
<code>invoke(input: List[ComponentResultObject]) -> List[ComponentResultObject]</code>	Generates embeddings for input text.

Input Fields

Field	Type	Description
<code>content.original_text</code>	str	Text to embed

Output Fields

Field	Type	Description
<code>content.embedding</code>	list	Vector embedding of the input text
<code>preprocessing.model</code>	str	Model used for embedding

ComparatorComponent

```
class ComparatorComponent(BaseComponent)
```

Component for comparing and ranking content.

Methods

Method	Description
<code>__init__()</code>	Initializes a new ComparatorComponent.
<code>invoke(input: List[ComponentResultObject]) -> List[ComponentResultObject]</code>	Compares and ranks input data.

Input Fields

Field	Type	Description
content.original_text	str	Source text for comparison
preprocessing.result_text	str	Target text for comparison

Output Fields

Field	Type	Description
preprocessing.score	float	Similarity score between texts
preprocessing.summary	str	Summary of comparison results

Functions Module

The functions module integrates with external AI services and provides specific implementations.

Base Implementations

BaseChat

```
class BaseChat(ABC)
```

Abstract base class for chat model implementations.

Methods

Method	Description
__init__(model: str = None)	Initializes a new BaseChat with optional model.
chat(prompt: str, history: list = None) -> str	Abstract method for chat interaction.

BaseEmbedding

```
class BaseEmbedding(ABC)
```

Abstract base class for embedding model implementations.

Methods

Method	Description
<code>__init__(model: str = None)</code>	Initializes a new BaseEmbedding with optional model.
<code>embed(text: str) -> list</code>	Abstract method for generating embeddings.

BaseReranker

```
class BaseReranker(ABC)
```

Abstract base class for reranking model implementations.

Methods

Method	Description
<code>__init__(model: str = None)</code>	Initializes a new BaseReranker with optional model.
<code>rerank(query: str, documents: List[str]) -> List[dict]</code>	Abstract method for reranking documents.

Provider Implementations

OpenAI

The OpenAI implementations provide integration with OpenAI's models.

OpenAIChat

```
class OpenAIChat(BaseChat)
```

Implementation of chat functionality using OpenAI models.

Methods

Method	Description
<code>__init__(model: str = "gpt-3.5-turbo")</code>	

Method	Description
	Initializes a new OpenAIChat with specified model.
<code>chat(prompt: str, history: list = None) -> str</code>	Interacts with OpenAI chat models.

OpenAIEmbedding

```
class OpenAIEmbedding(BaseEmbedding)
```

Implementation of embedding functionality using OpenAI models.

Methods

Method	Description
<code>__init__(model: str = "text-embedding-3-small")</code>	Initializes a new OpenAIEmbedding with specified model.
<code>embed(text: str) -> list</code>	Generates embeddings using OpenAI models.

HuggingFace

The HuggingFace implementations provide integration with HuggingFace's models.

HuggingFaceChat

```
class HuggingFaceChat(BaseChat)
```

Implementation of chat functionality using HuggingFace models.

Methods

Method	Description
<code>__init__(model: str = "mistralai/Mistral-7B-Instruct-v0.2")</code>	Initializes a new HuggingFaceChat with specified model.
<code>chat(prompt: str, history: list = None) -> str</code>	Interacts with HuggingFace chat models.

HuggingFaceEmbedding

```
class HuggingFaceEmbedding(BaseEmbedding)
```

Implementation of embedding functionality using HuggingFace models.

Methods

Method	Description
<code>__init__(model: str = "sentence-transformers/all-MiniLM-L6-v2")</code>	Initializes a new HuggingFaceEmbedding with specified model.
<code>embed(text: str) -> list</code>	Generates embeddings using HuggingFace models.

Ollama

The Ollama implementations provide integration with locally-hosted Ollama models.

OllamaChat

```
class OllamaChat(BaseChat)
```

Implementation of chat functionality using Ollama models.

Methods

Method	Description
<code>__init__(model: str = "llama2")</code>	Initializes a new OllamaChat with specified model.
<code>chat(prompt: str, history: list = None) -> str</code>	Interacts with Ollama chat models.

OllamaEmbedding

```
class OllamaEmbedding(BaseEmbedding)
```

Implementation of embedding functionality using Ollama models.

Methods

Method	Description
<code>__init__(model: str = "nomic-embed-text")</code>	Initializes a new OllamaEmbedding with specified model.
<code>embed(text: str) -> list</code>	Generates embeddings using Ollama models.

Additional Services

ElevenLabsAudio

```
class ElevenLabsAudio
```

Provides text-to-speech capabilities using ElevenLabs.

Methods

Method	Description
<code>__init__(voice_id: str = None)</code>	Initializes a new ElevenLabsAudio with optional voice ID.
<code>text_to_speech(text: str, output_path: str) -> str</code>	Converts text to speech and saves to file.

API Module

The API module provides interfaces for external communication and data management.

Cro (Component Result Object API)

```
class Cro
```

API for working with Component Result Objects, particularly for file operations.

Methods

Method	Description
<code>__init__()</code>	Initializes a new Cro instance.

Method	Description
<code>retrieve(input: List[ComponentResultObject]) -> List[ComponentResultObject]</code>	Reads data from sources specified in the input objects.
<code>write(input: List[ComponentResultObject]) -> bool</code>	Writes data to targets specified in the input objects.

Usage Example

```
# Initialize the CRO API
cro_api = Cro()

# Create input objects with source paths
inputs = [ComponentResultObject(source="/path/to/file.json")]

# Retrieve data
results = cro_api.retrieve(inputs)

# Write data
cro_api.write(results)
```

VectorDB

```
class VectorDB
```

Interface for vector database operations, supporting storage and retrieval of embeddings.

Methods

Method	Description
<code>__init__(client_path: str, client_collection: str)</code>	Initializes a new VectorDB with specified path and collection.
<code>retrieve(input: ComponentResultObject) -> List[ComponentResultObject]</code>	Retrieves similar documents based on input embedding.
<code>write(input: List[ComponentResultObject]) -> bool</code>	Stores embeddings in the vector database.

Input Fields (for retrieve)

Field	Type	Description
content.embedding	list	Vector embedding to use for similarity search
preprocessing.top_k	int	Number of results to return (optional)

Output Fields (from retrieve)

Field	Type	Description
content.original_text	str	Text content of retrieved document
content.embedding	list	Vector embedding of the document
preprocessing.score	float	Similarity score

Usage Example

```
# Initialize the vector database
vdb = VectorDB(client_path="/path/to/db", client_collection="documents")

# Store embeddings
vdb.write(embedded_documents)

# Retrieve similar documents
results = vdb.retrieve(query_embedding)
```

ServiceIntegration

```
class ServiceIntegration
```

Integration with external services and APIs.

Methods

Method	Description
<code>__init__(service_name: str, credentials: dict = None)</code>	Initializes a new ServiceIntegration for the specified service.
<code>connect() -> bool</code>	Establishes connection to the service.

Method	Description
<code>execute(operation: str, parameters: dict)</code> <code>-> Any</code>	Executes an operation on the service.
<code>disconnect() -> bool</code>	Closes the connection to the service.

Agents Module

The Agents module implements autonomous agents that can perform tasks and make decisions.

BaseAgent

```
class BaseAgent(ABC)
```

Abstract base class for all agents.

Methods

Method	Description
<code>__init__(name: str, capabilities: List[str] = None)</code>	Initializes a new BaseAgent with a name and capabilities.
<code>process(task: str) -> dict</code>	Abstract method that processes a task and returns results.
<code>add_capability(capability: str) -> bool</code>	Adds a new capability to the agent.

TaskPlanner

```
class TaskPlanner
```

Plans and schedules tasks for agents.

Methods

Method	Description
<code>__init__()</code>	Initializes a new TaskPlanner.

Method	Description
<code>create_plan(task: str) -> List[dict]</code>	Creates a plan of subtasks for the given task.
<code>prioritize(tasks: List[dict]) -> List[dict]</code>	Prioritizes tasks based on importance and dependencies.
<code>assign(tasks: List[dict], agents: List[BaseAgent]) -> dict</code>	Assigns tasks to appropriate agents.

Executor

```
class Executor
```

Executes agent tasks and actions.

Methods

Method	Description
<code>__init__(agent: BaseAgent)</code>	Initializes a new Executor for the specified agent.
<code>execute(task: dict) -> dict</code>	Executes a task and returns the result.
<code>monitor() -> dict</code>	Monitors the execution status.
<code>abort() -> bool</code>	Aborts the current execution.

MultiAgent

```
class MultiAgent
```

Coordinates multiple agents for collaborative tasks.

Methods

Method	Description
<code>__init__(agents: List[BaseAgent])</code>	Initializes a new MultiAgent with a list of agents.

Method	Description
<code>coordinate(task: str) -> dict</code>	Coordinates agents to complete a complex task.
<code>communicate(from_agent: str, to_agent: str, message: dict) -> bool</code>	Facilitates communication between agents.
<code>aggregate_results(results: List[dict]) -> dict</code>	Aggregates results from multiple agents.

Memory

```
class Memory
```

Manages agent memory and knowledge.

Methods

Method	Description
<code>__init__(capacity: int = None)</code>	Initializes a new Memory with optional capacity limit.
<code>store(key: str, value: Any) -> bool</code>	Stores information in memory.
<code>retrieve(key: str) -> Any</code>	Retrieves information from memory.
<code>forget(key: str) -> bool</code>	Removes information from memory.
<code>summarize() -> dict</code>	Generates a summary of stored information.

Applications Module

The applications module contains practical implementations and examples.

MCP-Server

The MCP-Server demonstrates function call standardization using the Model Context Protocol.

server.py

```
class FastMCP
```

Implementation of the Model Context Protocol server.

Methods

Method	Description
<code>__init__(name: str)</code>	Initializes a new FastMCP server with a name.
<code>tool()</code>	Decorator for registering tools with the server.
<code>resource(path: str)</code>	Decorator for registering resources with the server.

Usage Example

```
# Create an MCP server
mcp = FastMCP("Demo")

# Add a tool
@mcp.tool()
def add(a: int, b: int) -> int:
    """Add two numbers"""
    return a + b

# Add a resource
@mcp.resource("greeting://{name}")
def get_greeting(name: str) -> str:
    """Get a personalized greeting"""
    return f"Hello, {name}!"
```

examples.py

The examples.py file contains various usage examples for the YaniG framework. See the Examples & Use Cases section for detailed explanations of these examples.

5. Examples & Use Cases

This section provides detailed examples and use cases for the YaniG framework, demonstrating how to use its components for various tasks. These examples are based

on the `examples.py` file in the repository and are expanded with additional context and explanations.

PDF Reader Example

This example demonstrates how to use the `PDFReaderComponent` to extract and process content from PDF files.

```
def components_pdfreadercomponent_retrieve(app_path):
    # Define the path to the PDF file
    pdf_path = join(app_path, 'data\\pdf\\beispiel.pdf')

    # Initialize the PDF reader component
    pdfreader = PDFReaderComponent()

    # Create input with source path and page settings
    input = ComponentResultObject()
    input["source"] = pdf_path
    input["content"] = {}
    input["content"]["page_number"] = 1
    input["content"]["page_count"] = 1

    # Process the PDF and get results
    results = pdfreader.invoke([input])

    # Print the extracted content
    for result in results:
        print(result["content"]["chapter"], result["content"]["page_number"])
        print(result["content"]["original_text"])
```

How It Works

1. The example first defines the path to a PDF file.
2. It initializes a `PDFReaderComponent` instance.
3. It creates a `ComponentResultObject` with the source path and page settings.
4. It invokes the component with the input object.
5. It iterates through the results and prints the extracted content.

Use Cases

- **Document Analysis:** Extract text from PDF documents for further processing.
- **Content Indexing:** Build searchable indexes from PDF content.
- **Information Extraction:** Extract specific information from structured PDF documents.

- **Data Migration:** Convert PDF content to other formats for data migration.

Embedding Generation Example

This example demonstrates how to generate embeddings from text and save them to a JSON file.

```
def components_embeddingcomponent_api_cro_write(app_path):  
    # Initialize the embedding component  
    ec = EmbeddingComponent()  
  
    # Initialize the CRO API client  
    cro_api = Cro()  
  
    # Read example text from a file  
    txt_path = join(app_path, 'data\\txt\\example.txt')  
    with open(txt_path, "r", encoding="utf-8") as f:  
        info = None  
        for l in f.readlines():  
            if '#' in l:  
                if info is not None:  
                    infos.append(info)  
                    info = ComponentResultObject()  
                    info["preprocessing"] = {}  
                    info["preprocessing"]["category"] = l[2:-1]  
                    info["source"] = f"{l[2:-1]}.json"  
                    info["target"] = join(json_path, f"{l[2:-1]}.json")  
                    info["content"] = {}  
                    info["content"]["original_text"] = ""  
                else:  
                    if info is not None:  
                        info["content"]["original_text"] += l  
            else:  
                # Generate embeddings for the text  
                infos = ec.invoke(infos)  
  
        # Write the embeddings to a JSON file  
        cro_api.write(infos)
```

How It Works

1. The example initializes an `EmbeddingComponent` instance and a `Cro` API client.
2. It reads text from a file, parsing it into sections based on headings.
3. For each section, it creates a `ComponentResultObject` with the text content.
4. It generates embeddings for all sections by invoking the embedding component.
5. It writes the embeddings to JSON files using the CRO API.

Use Cases

- **Semantic Search:** Create embeddings for documents to enable semantic search.
- **Content Similarity:** Compare documents or sections based on their embeddings.
- **Classification:** Use embeddings as features for classification tasks.
- **Recommendation Systems:** Build recommendation systems based on content similarity.

Vector Database Example

This example demonstrates how to read JSON files, store their content in a vector database, and perform retrieval.

```
def api_cro_read_api_vectordb_write(app_path):  
    # Initialize the CRO API client and vector database  
    cro_api = Cro()  
    vdb = VectorDB(  
        client_path=join(app_path, 'data\\vdb\\wiki'),  
        client_collection="wiki"  
    )  
  
    # Read JSON files from a directory  
    json_path = join(app_path, 'data\\json\\wiki')  
    json_files = []  
    filenames = [f for f in listdir(json_path) if f.endswith(".json")]  
  
    # Process each JSON file  
    for filename in filenames:  
        # Create a ComponentResultObject for the file  
        cro_data = ComponentResultObject()  
        cro_data["source"] = join(json_path, filename)  
  
        # Add the file to the list  
        json_files.append(cro_data)  
  
    # Read the JSON files using the CRO API  
    results = cro_api.retrieve(json_files)  
  
    # Write the results to the vector database  
    vdb.write(results)
```

How It Works

1. The example initializes a `Cro` API client and a `VectorDB` instance.
2. It identifies JSON files in a directory.
3. For each file, it creates a `ComponentResultObject` with the file path.

4. It retrieves the content of all files using the CRO API.
5. It writes the retrieved data to the vector database.

Use Cases

- **Knowledge Base:** Build a searchable knowledge base from structured data.
- **Document Retrieval:** Enable efficient retrieval of documents based on semantic similarity.
- **Question Answering:** Support question answering systems with relevant document retrieval.
- **Content Organization:** Organize and retrieve content based on semantic relationships.

Comparator Example

This example demonstrates how to compare text in different languages using the `ComparatorComponent` .

```
def components_comparator_invoke(app_path):
    # Initialize the comparator component
    comparator = ComparatorComponent()

    # Prepare data for comparison
    datas = []

    # Read German text
    with open(join(app_path, 'data\\txt\\ger.txt'), encoding="utf-8") as f:
        data_ger = f.readlines()

    # Read English text
    with open(join(app_path, 'data\\txt\\eng.txt'), encoding="utf-8") as f:
        data_eng = f.readlines()

    # Check if the texts have the same length
    if len(data_ger) != len(data_eng):
        raise TypeError("length mismatch")

    # Process each line pair
    for i in range(len(data_eng)):
        if len(data_eng[i]) > 10:
            # Create a ComponentResultObject for the pair
            data = ComponentResultObject()
            data["content"] = {}
            data["content"]["original_text"] = data_eng[i]
            data["preprocessing"] = {}
            data["preprocessing"]["result_text"] = data_ger[i]
            datas.append(data)
```

```

# Invoke the comparator component
print("Invoke ComparatorComponent")
result = comparator.invoke(datas)

# Write the results to a file
with open(join(app_path, 'data\\txt\\res.txt'), "w", encoding="utf-8") as f:
    for res in result:
        f.write("---\n")
        f.write(str(res["preprocessing"]["score"]))
        f.write("\n")
        f.write(res["preprocessing"]["summary"])
        f.write("\n\n")
        f.write(res["content"]["original_text"])
        f.write("\n")
        f.write(res["preprocessing"]["result_text"])
        f.write("\n")
        f.write("\n")

```

How It Works

1. The example initializes a `ComparatorComponent` instance.
2. It reads German and English text files.
3. It creates `ComponentResultObject` instances for each pair of corresponding lines.
4. It invokes the comparator component to compare the text pairs.
5. It writes the comparison results to a file, including similarity scores and summaries.

Use Cases

- **Translation Evaluation:** Assess the quality of translations between languages.
- **Content Alignment:** Align corresponding content in different languages.
- **Plagiarism Detection:** Identify similar content across different sources.
- **Version Comparison:** Compare different versions of the same document.

Semantic Search Example

This example demonstrates how to perform semantic search using embeddings and a vector database.

```

def components_embeddingcomponent_api_vektordb_read(app_path):
    # Initialize the embedding component and vector database
    ec = EmbeddingComponent()
    vdb = VectorDB(
        client_path=join(app_path, 'data\\vdb\\wiki'),
        client_collection="wiki"
    )

```

```

)

# Create a query
question = ComponentResultObject()
question["content"] = {}
question["content"]["original_text"] = "Welche neuen Technologien gibt es, die Menschen unterstützen?"

# Generate embedding for the query
question = ec.invoke([question])[0]

# Perform semantic search
results = vdb.retrieve(question)

# Process and print the results
for result in results:
    print(f"Score: {result['preprocessing']['score']}")
    print(f"Content: {result['content']['original_text']}")

```

How It Works

1. The example initializes an `EmbeddingComponent` instance and a `VectorDB` instance.
2. It creates a query as a `ComponentResultObject`.
3. It generates an embedding for the query using the embedding component.
4. It retrieves semantically similar documents from the vector database.
5. It prints the retrieved documents along with their similarity scores.

Use Cases

- **Question Answering:** Find relevant information to answer user questions.
- **Information Retrieval:** Retrieve documents based on semantic meaning rather than keyword matching.
- **Research Assistance:** Support research by finding relevant documents on a topic.
- **Content Discovery:** Help users discover content related to their interests.

Chat Component Example

This example demonstrates how to use the `ChatComponent` for interactive conversations.

```

def components_chatcomponent():
    # Initialize the chat component
    chat = ChatComponent()

```

```
# Start a conversation
```

```
response = chat.invoke("Tell me about natural language processing")  
print(response)
```

```
# Continue the conversation
```

```
follow_up = chat.invoke("What are some practical applications?")  
print(follow_up)
```

How It Works

1. The example initializes a `ChatComponent` instance.
2. It starts a conversation by invoking the component with an initial prompt.
3. It continues the conversation with a follow-up question.
4. The component maintains conversation context between interactions.

Use Cases

- **Conversational Interfaces:** Build chatbots or conversational agents.
- **Interactive Documentation:** Create interactive documentation that responds to user queries.
- **Educational Tools:** Develop educational tools that engage in dialogue with students.
- **Customer Support:** Implement automated customer support systems.

MCP-Server Example

This example demonstrates how to use the Model Context Protocol server for standardized function calls.

```
# Create an MCP server
```

```
mcp = FastMCP("Demo")
```

```
# Add an addition tool
```

```
@mcp.tool()
```

```
def add(a: int, b: int) -> int:
```

```
    """Add two numbers"""
```

```
    return a + b
```

```
# Add a greeting resource
```

```
@mcp.resource("greeting://{name}")
```

```
def get_greeting(name: str) -> str:
```

```
    """Get a personalized greeting"""
```

```
    return f"Hello, {name}!"
```

```
# Run the server
```

```
# To connect and query tools and resources:  
# mcp dev server.py
```

How It Works

1. The example creates a `FastMCP` server instance.
2. It defines a tool for adding numbers using the `@mcp.tool()` decorator.
3. It defines a resource for generating greetings using the `@mcp.resource()` decorator.
4. The server can be run and queried to access these tools and resources.

Use Cases

- **Function Calling:** Standardize function calls from language models.
- **API Abstraction:** Create a consistent interface for various functions.
- **Tool Integration:** Integrate tools and resources with language models.
- **Service Orchestration:** Orchestrate multiple services through a unified interface.

Additional Use Cases

Beyond the examples provided in the repository, YaniG can be applied to various other use cases:

Document Processing Pipeline

Combine multiple components to create a document processing pipeline:

```
# Initialize components  
pdf_reader = PDFReaderComponent()  
embedding = EmbeddingComponent()  
vdb = VectorDB(client_path="data/vdb/documents", client_collection="docs")  
  
# Create a processing pipeline  
def process_document(pdf_path):  
    # Extract content from PDF  
    input_obj = ComponentResultObject()  
    input_obj["source"] = pdf_path  
    input_obj["content"] = {"page_number": 1, "page_count": -1} # Process all pages  
  
    # Extract text from PDF  
    results = pdf_reader.invoke([input_obj])  
  
    # Generate embeddings for each page  
    embedded_results = embedding.invoke(results)
```

```
# Store in vector database
vdb.write(embedded_results)

return len(embedded_results)
```

Multimodal Content Generation

Combine text generation with audio synthesis:

```
# Initialize components
chat = ChatComponent()
audio = ElevenLabsAudio()

# Create a multimodal generation function
def generate_content(prompt, output_audio_path):
    # Generate text response
    text_response = chat.invoke(prompt)

    # Convert to audio
    audio_path = audio.text_to_speech(text_response, output_audio_path)

    return {
        "text": text_response,
        "audio": audio_path
    }
```

Comparative Analysis System

Build a system for comparing and analyzing multiple documents:

```
# Initialize components
pdf_reader = PDFReaderComponent()
embedding = EmbeddingComponent()
comparator = ComparatorComponent()

# Create an analysis function
def compare_documents(doc_paths):
    # Extract content from all documents
    contents = []
    for path in doc_paths:
        input_obj = ComponentResultObject()
        input_obj["source"] = path
        input_obj["content"] = {"page_number": 1, "page_count": -1}
        results = pdf_reader.invoke([input_obj])
        contents.extend(results)

    # Generate embeddings
    embedded_contents = embedding.invoke(contents)
```

```

# Compare all pairs
comparisons = []
for i in range(len(embedded_contents)):
    for j in range(i+1, len(embedded_contents)):
        comp_input = ComponentResultObject()
        comp_input["content"] = {"original_text": embedded_contents[i]["content"]}
        comp_input["preprocessing"] = {"result_text": embedded_contents[j]
        ["content"]["original_text"]}
        comparisons.append(comp_input)

# Get comparison results
comparison_results = comparator.invoke(comparisons)

return comparison_results

```

These additional use cases demonstrate the flexibility and extensibility of the YaniG framework for various applications involving AI components and workflows.

6. FAQ / Troubleshooting

This section addresses common questions and issues that users might encounter when working with the YaniG framework. It provides solutions and workarounds for typical problems.

Installation Issues

Q: I'm getting dependency conflicts when installing YaniG. How can I resolve them?

A: Dependency conflicts are common when working with AI libraries that have specific version requirements. Try the following solutions:

1. Create a fresh virtual environment before installation: `bash python -m venv yanig_env source yanig_env/bin/activate` # On Windows: `yanig_env\Scripts\activate`
2. Install dependencies in the correct order: `bash pip install -r requirements.txt`
3. If conflicts persist, try installing problematic dependencies individually with specific versions.

Q: How do I resolve "Module not found" errors after installation?

A: This typically occurs when the Python interpreter can't find the YaniG modules. Ensure that:

1. You're running your code from the project root directory.
2. The virtual environment where YaniG is installed is activated.
3. If using an IDE, configure it to use the correct Python interpreter.
4. If needed, add the YaniG directory to your Python path: `python import sys
sys.path.append('/path/to/yanig')`

API Key Configuration

Q: How do I set up API keys for the different providers?

A: YaniG uses environment variables for API keys. You can set them up in several ways:

1. Create a `.env` file in the project root with your API keys:
`OPENAI_API_KEY=your_openai_key
HUGGINGFACEHUB_API_TOKEN=your_huggingface_token
ELEVENLABS_API_KEY=your_elevenlabs_key`
2. Set environment variables directly in your terminal: ```` bash # Linux/macOS
export OPENAI_API_KEY=your_openai_key`

Windows set `OPENAI_API_KEY=your_openai_key` ```

1. Set them programmatically in your Python code (not recommended for production): `python import os os.environ["OPENAI_API_KEY"] =
"your_openai_key"`

Q: I'm getting authentication errors when using AI services. What should I check?

A: Authentication errors usually indicate issues with API keys:

1. Verify that your API keys are correct and not expired.
2. Check that the keys are properly set in your environment.
3. Ensure you have sufficient credits or quota with the service provider.
4. Check if your IP address is allowed by the service provider.
5. Verify that you're using the correct API endpoint for your subscription tier.

Component Usage

Q: Why am I getting empty results from the PDFReaderComponent?

A: Empty results from the PDFReaderComponent can occur for several reasons:

1. The PDF file path is incorrect or the file doesn't exist.
2. The PDF is encrypted or password-protected.
3. The PDF contains scanned images rather than text (OCR might be needed).
4. The page number settings are outside the range of the document.

Try the following solutions: - Verify the file path and existence. - Check if the PDF contains actual text (not just images). - Ensure page numbers are within the document's range. - Try a different PDF to see if the issue is file-specific.

Q: How can I debug component chains when they're not working as expected?

A: Debugging component chains can be challenging. Here are some strategies:

1. Test each component individually before chaining them.
2. Add print statements to inspect the input and output at each step:

```
python for i, component in enumerate(components): print(f"Before component {i}:", input_data) input_data = component.invoke(input_data) print(f"After component {i}:", input_data)
```
3. Create a simple debugging wrapper:

```
python class DebugComponent(BaseComponent): def init(self, component, name): self.component = component self.name = name def invoke(self, input): print(f"[DEBUG] {self.name} input:", input) result = self.component.invoke(input) print(f"[DEBUG] {self.name} output:", result) return result
```

```
# Usage debug_chain = ComponentChain([ DebugComponent(pdf_reader, "PDFReader"), DebugComponent(embedding, "Embedding") ]) ````
```

Model-Specific Issues

Q: The chat component is generating irrelevant or incorrect responses. How can I improve them?

A: Improving chat responses can be done in several ways:

1. Provide a better system prompt to guide the model:

```
python input_obj = ComponentResultObject() input_obj["preprocessing"] = {"system_prompt": "You are a helpful assistant that provides accurate information about AI."} input_obj["content"] = {"original_text": "Explain neural networks"}
```
2. Try a different model or provider:

```
python chat = ChatComponent(model="gpt-4", provider="openai") # or chat = ChatComponent(model="llama2", provider="ollama")
```
3. Implement post-processing to filter or enhance responses.
4. Use a more structured prompt format with examples.

Q: Embedding generation is slow. How can I optimize it?

A: Embedding generation can be optimized in several ways:

1. Use a smaller, faster embedding model:

```
python embedding = EmbeddingComponent(model="all-MiniLM-L6-v2")
```
2. Batch your requests instead of processing one at a time:

```
python # Instead of: for text in texts: embedding.invoke([text])
```

```
# Do: all_inputs = [ComponentResultObject(content={"original_text": text}) for text in texts] all_embeddings = embedding.invoke(all_inputs) ````
```

1. Consider using local models via Ollama for lower latency:

```
python embedding = EmbeddingComponent(provider="ollama", model="nomic-embed-text")
```
2. Cache embeddings for frequently used content.

MCP-Server Issues

Q: The MCP-Server isn't starting. What could be wrong?

A: If the MCP-Server fails to start, check the following:

1. Ensure Node.js is installed and accessible from your command line: `bash node --version`
2. Verify that all dependencies are installed: `bash cd applications/mcpserver npm install`
3. Check for port conflicts. The server might be trying to use a port that's already in use.
4. Look for error messages in the console output for specific issues.
5. Ensure you have the necessary permissions to start a server on your system.

Q: How do I connect to the MCP-Server from my application?

A: To connect to the MCP-Server:

1. Start the server: `bash cd applications/mcpserver python server.py`
2. Use the MCP client to connect: `python from mcp.client import MCPClient`

```
client = MCPClient("http://localhost:8000")
```

```
# Call a tool result = client.call_tool("add", {"a": 5, "b": 3}) print(result) # Output: 8
```

```
# Access a resource greeting = client.get_resource("greeting://Alice") print(greeting) #  
Output: "Hello, Alice!"
```

1. If connecting from a different machine, replace "localhost" with the server's IP address.

Data Handling

Q: How should I structure my data directory for YaniG?

A: YaniG expects a specific directory structure for optimal operation:

```
yanig/  
├── data/
```

		pdf/	# PDF documents
		txt/	# Text files
		json/	# JSON data
		vdb/	# Vector database files
		audio/	# Audio output files

Create this structure in your project directory and place files in the appropriate subdirectories.

Q: I'm getting encoding errors when reading text files. How can I fix this?

A: Encoding errors are common when working with text files. To fix them:

1. Always specify the encoding when opening files: `python with open(file_path, "r", encoding="utf-8") as f: content = f.read()`
2. If you know the file uses a different encoding, specify it: `python with open(file_path, "r", encoding="latin-1") as f: content = f.read()`
3. For files with unknown encoding, you can try to detect it: `python import chardet`

```
with open(file_path, "rb") as f: raw_data = f.read()
result = chardet.detect(raw_data)
encoding = result["encoding"]
```

```
with open(file_path, "r", encoding=encoding) as f: content = f.read()
```

Performance Optimization

Q: YaniG is using too much memory. How can I reduce memory usage?

A: To reduce memory usage:

1. Process data in smaller batches: `python # Instead of processing all data at once`
`results = component.invoke(all_inputs)`

```
# Process in batches
batch_size = 10
all_results = []
for i in range(0, len(all_inputs), batch_size):
    batch = all_inputs[i:i+batch_size]
    batch_results = component.invoke(batch)
    all_results.extend(batch_results)
```

1. Release resources when not needed: `python import gc`

```
# After processing large data large_data = None gc.collect() ```
```

1. Use smaller models when possible: ```python # Instead of chat = ChatComponent(model="gpt-4")

```
# Use chat = ChatComponent(model="gpt-3.5-turbo") ```
```

1. Optimize your data structures to avoid redundant storage.

Q: How can I speed up processing for large documents?

A: For large documents:

1. Process documents in parallel: ```python from concurrent.futures import ThreadPoolExecutor

```
def process_document(doc_path): # Processing logic here return result
```

```
with ThreadPoolExecutor(max_workers=4) as executor: results = list(executor.map(process_document, document_paths)) ```
```

1. Split large documents into smaller chunks before processing.
2. Use more efficient models or local models for faster inference.
3. Implement caching for repeated operations.
4. Consider preprocessing documents offline and storing the results.

Integration with Other Systems

Q: How can I integrate YaniG with a web application?

A: To integrate YaniG with a web application:

1. Create a simple API wrapper: ```python from flask import Flask, request, jsonify from components import ChatComponent

```
app = Flask(name) chat = ChatComponent()
```

```
@app.route("/chat", methods=["POST"]) def chat_endpoint(): data = request.json prompt = data.get("prompt", "") response = chat.invoke(prompt) return jsonify({"response": response})
```

```
if name == "main": app.run(debug=True) ```
```

1. Use YaniG as a backend service that your web application calls.
2. Consider containerizing YaniG with Docker for easier deployment.

3. Implement proper error handling and rate limiting for production use.

Q: Can YaniG be used in a production environment?

A: YaniG is primarily designed for educational purposes, but it can be adapted for production with some considerations:

1. Implement proper error handling and logging.
2. Add authentication and authorization for API endpoints.
3. Set up monitoring and alerting for system health.
4. Consider scaling strategies for handling increased load.
5. Implement caching and optimization for performance.
6. Ensure compliance with relevant regulations for AI systems.
7. Develop a comprehensive testing strategy.

For critical production systems, you might want to adapt YaniG's architecture rather than using it directly.

Additional Resources

If you encounter issues not covered in this FAQ, consider these additional resources:

1. Check the GitHub repository issues section for similar problems.
2. Consult the documentation for the specific AI services you're using (OpenAI, HuggingFace, etc.).
3. Search for similar issues in AI and machine learning forums.
4. Consider contributing to YaniG by reporting issues and suggesting improvements.

7. Contributing / Development

This section provides guidelines for developers who want to contribute to the YaniG project or extend it for their own purposes. It covers development setup, coding standards, and extension points.

Development Setup

Prerequisites

To set up a development environment for YaniG, you'll need:

- Python 3.12.x or higher

- Git
- A code editor or IDE (e.g., Visual Studio Code, PyCharm)
- Node.js (for MCP-Server development)

Setting Up the Development Environment

1. **Clone the repository:** `bash git clone https://github.com/spie-mkroehn/yanig.git`
`cd yanig`
2. **Create a virtual environment:** `bash python -m venv venv source venv/bin/activate` # On Windows: `venv\Scripts\activate`
3. **Install dependencies:** `bash pip install -r requirements.txt`
4. **Install development dependencies:** `bash pip install pytest black mypy ruff sphinx`
5. **Set up environment variables:** Create a `.env` file in the project root with your API keys and configuration: `OPENAI_API_KEY=your_openai_api_key`
`HUGGINGFACEHUB_API_TOKEN=your_huggingface_token`
`ELEVENLABS_API_KEY=your_elevenlabs_key`

Project Structure

Understanding the project structure is essential for effective development:

```
yanig/
├── agents/           # Agent implementations
├── api/             # API interfaces
├── applications/     # Example applications
│   ├── mcpserver/    # Model Context Protocol server
│   └── examples.py   # Example usage patterns
├── components/       # Component implementations
│   ├── __init__.py
│   ├── basecomponent.py # Base component class
│   └── ...           # Specific components
├── core/             # Core functionality
│   ├── __init__.py
│   ├── componentresultobject.py
│   └── ...           # Core modules
├── data/             # Data directory
├── functions/        # Model integrations
│   ├── __init__.py
│   ├── basechat.py   # Base chat implementation
│   └── ...           # Provider-specific implementations
```

```
|—— .gitignore
|—— LICENSE
|—— README.md
|—— main.py          # Main entry point
|—— requirements.txt  # Dependencies
```

Coding Standards

YaniG follows these coding standards to maintain code quality and consistency:

Python Style Guide

- Follow [PEP 8](#) for code style.
- Use [Black](#) for code formatting: `bash black .`
- Use [Ruff](#) for linting: `bash ruff check .`
- Use [MyPy](#) for type checking: `bash mypy .`

Documentation

- Use docstrings for all modules, classes, and functions.
- Follow [Google-style docstrings](#) format: ````python def function(arg1, arg2):
 """Short description of the function.`

Longer description explaining the function in detail.

Args: arg1: Description of arg1 arg2: Description of arg2

Returns: Description of return value

Raises: ExceptionType: When and why this exception is raised """ pass ``` - Keep documentation up-to-date with code changes. - Use inline comments sparingly and only when necessary to explain complex logic.

Commit Messages

- Use clear, descriptive commit messages.
- Follow the format: `[Component] Brief description of change`
- For example: `[PDFReader] Fix page number handling for multi-page documents`

Testing

YaniG uses pytest for testing. Follow these guidelines for testing:

Writing Tests

- Create test files in a `tests` directory with names matching `test_*.py`.
- Write tests for all new functionality.
- Use fixtures to set up test environments.
- Mock external services to avoid API calls during testing.

Running Tests

```
# Run all tests  
pytest  
  
# Run tests for a specific module  
pytest tests/test_components.py  
  
# Run tests with coverage report  
pytest --cov=yanig
```

Test Coverage

- Aim for at least 80% test coverage for new code.
- Focus on testing edge cases and error conditions.
- Use parameterized tests for testing multiple input variations.

Pull Request Process

When contributing to YaniG, follow this pull request process:

1. **Fork the repository** and create a new branch for your feature or fix.
2. **Implement your changes**, following the coding standards.
3. **Add tests** for your changes to ensure they work as expected.
4. **Run the test suite** to make sure all tests pass.
5. **Update documentation** to reflect your changes.
6. **Submit a pull request** with a clear description of the changes.
7. **Address review comments** and make necessary adjustments.
8. Once approved, your changes will be merged into the main branch.

Extension Points

YaniG is designed to be extensible. Here are the main extension points:

Creating New Components

To create a new component, extend the `BaseComponent` class:

```
from components import BaseComponent
from core import ComponentResultObject
from typing import List

class MyCustomComponent(BaseComponent):
    """
    A custom component that does something useful.
    """
    def __init__(self, param1=None, param2=None):
        """Initialize the component with optional parameters."""
        self.param1 = param1
        self.param2 = param2

    def invoke(self, input: List[ComponentResultObject]) ->
List[ComponentResultObject]:
        """
        Process the input and return results.

        Args:
            input: List of ComponentResultObject instances

        Returns:
            List of processed ComponentResultObject instances
        """
        results = []
        for item in input:
            # Create a new result object
            result = ComponentResultObject()

            # Copy source and other metadata
            result["source"] = item.get("source", "")

            # Add your processing logic here
            result["content"] = {}
            result["content"]["original_text"] = item.get("content", {}).get("original_text",
            "")

            result["content"]["processed_text"] = self._process_text(result["content"]
            ["original_text"])

            results.append(result)

        return results

    def _process_text(self, text):
        """Internal method for text processing."""
```

```
# Your processing logic here
return text.upper() # Example: convert to uppercase
```

Integrating New AI Models

To integrate a new AI model provider, create appropriate implementations in the `functions` module:

```
from functions import BaseChat
from typing import List, Optional

class NewProviderChat(BaseChat):
    """
    Chat implementation for a new provider.
    """
    def __init__(self, model: str = "default-model"):
        """Initialize with the specified model."""
        self.model = model
        # Add any provider-specific initialization

    def chat(self, prompt: str, history: Optional[List] = None) -> str:
        """
        Generate a chat response.

        Args:
            prompt: The user prompt
            history: Optional conversation history

        Returns:
            Generated response text
        """
        # Implement provider-specific chat logic
        # This might involve API calls, local model inference, etc.
        return "Response from new provider"
```

Creating Custom Applications

You can create custom applications using YaniG components:

```
from components import PDFReaderComponent, EmbeddingComponent,
ChatComponent
from core import ComponentResultObject, ComponentChain
from os.path import join

class DocumentQASystem:
    """
    A question-answering system for documents.
```

```

"""
def __init__(self, data_dir):
    """Initialize the system with a data directory."""
    self.data_dir = data_dir
    self.pdf_reader = PDFReaderComponent()
    self.embedding = EmbeddingComponent()
    self.chat = ChatComponent()

def load_document(self, filename):
    """Load and process a document."""
    input_obj = ComponentResultObject()
    input_obj["source"] = join(self.data_dir, filename)
    input_obj["content"] = {"page_number": 1, "page_count": -1}

    # Extract text and generate embeddings
    results = self.pdf_reader.invoke([input_obj])
    self.document_content = self.embedding.invoke(results)

def ask_question(self, question):
    """Ask a question about the loaded document."""
    # Create a prompt with document context
    context = "\n\n".join([doc["content"]["original_text"] for doc in
self.document_content])
    prompt = f"Based on the following document:\n\n{context}\n\nQuestion:
{question}\n\nAnswer:"

    # Get response from chat model
    response = self.chat.invoke(prompt)
    return response

```

Best Practices

Follow these best practices when developing with YaniG:

Component Design

- Keep components focused on a single responsibility.
- Make components configurable through constructor parameters.
- Document the expected input and output formats clearly.
- Handle errors gracefully and provide meaningful error messages.
- Consider performance implications, especially for large inputs.

Data Handling

- Use `ComponentResultObject` consistently for data exchange.
- Maintain the standard field structure (`source` , `content` , `preprocessing` , etc.).

- Be mindful of memory usage when processing large datasets.
- Implement proper cleanup for temporary resources.

Model Integration

- Abstract provider-specific details behind common interfaces.
- Handle API rate limits and errors appropriately.
- Provide fallback mechanisms when possible.
- Cache results when appropriate to reduce API calls.

Security Considerations

- Never hardcode API keys or credentials in the code.
- Validate and sanitize user inputs.
- Be cautious with executing dynamic code or commands.
- Consider privacy implications when processing user data.

Documentation Tools

YaniG uses [Sphinx](#) for generating documentation. To build the documentation:

1. Install Sphinx and extensions: `bash pip install sphinx sphinx-rtd-theme sphinx-autodoc-typehints`
2. Generate API documentation: `bash sphinx-apidoc -o docs/api yanig`
3. Build the documentation: `bash cd docs make html`
4. View the documentation: `bash # Open docs/_build/html/index.html in your browser`

Visualization Tools

For generating class diagrams and other visualizations:

- Use [pyreverse](#) (part of pylint): `bash pyreverse -o png -p YaniG yanig`
- Alternatively, use [PlantUML](#) for more customized diagrams.

Community Guidelines

When participating in the YaniG community:

- Be respectful and inclusive in all communications.
- Provide constructive feedback on issues and pull requests.
- Help others learn and grow by sharing knowledge.
- Report bugs and issues with detailed reproduction steps.
- Suggest improvements with clear rationales.

By following these guidelines, you'll help make YaniG a better framework for everyone.